

React Landing Page - AWS Amplify Deployment Documentation

Project Overview

This document describes the process of building and deploying the **React Landing Page** using **AWS Amplify**, along with troubleshooting steps for common deployment issues.

Project Stack

- React
 - Vite
 - HTML5
 - CSS3
 - JavaScript
 - Git & GitHub
 - AWS Amplify
-

Project Structure

```
react-landing-page/
├── src/
│   ├── components/
│   ├── styles/
│   ├── data/
│   ├── assets/
│   ├── App.jsx
│   ├── App.css
│   ├── index.css
│   └── main.jsx
├── public/
├── package.json
├── package-lock.json
├── vite.config.js
└── README.md
```

Step 1: Install Dependencies

Navigate to the project directory.

```
cd react-landing-page
```

Install all required packages.

```
npm install
```

Output:

```
added 74 packages  
found 0 vulnerabilities
```

Step 2: Build the Project Locally

Generate the production build.

```
npm run build
```

Expected Output:

```
vite build  
  
✓ built successfully
```

A new folder named **dist/** will be created.

Step 3: Push Project to GitHub

Initialize Git (if not already initialized).

```
git init
```

Add all project files.

```
git add .
```

Commit the changes.

```
git commit -m "Initial Commit"
```

Rename the branch.

```
git branch -M main
```

Connect the GitHub repository.

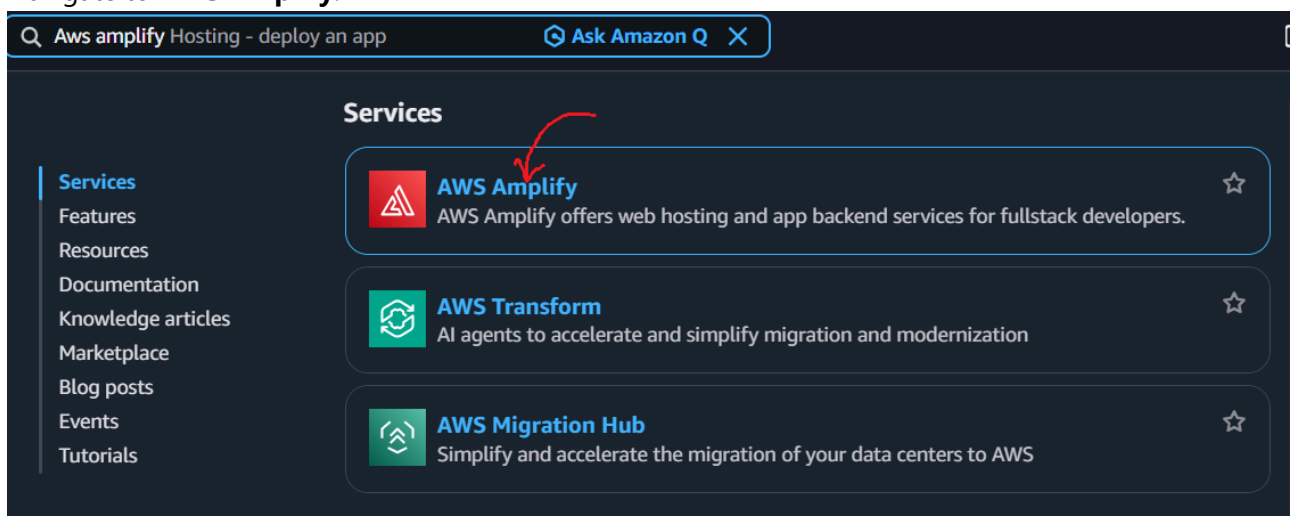
```
git remote add origin <repository-url>
```

Push the code.

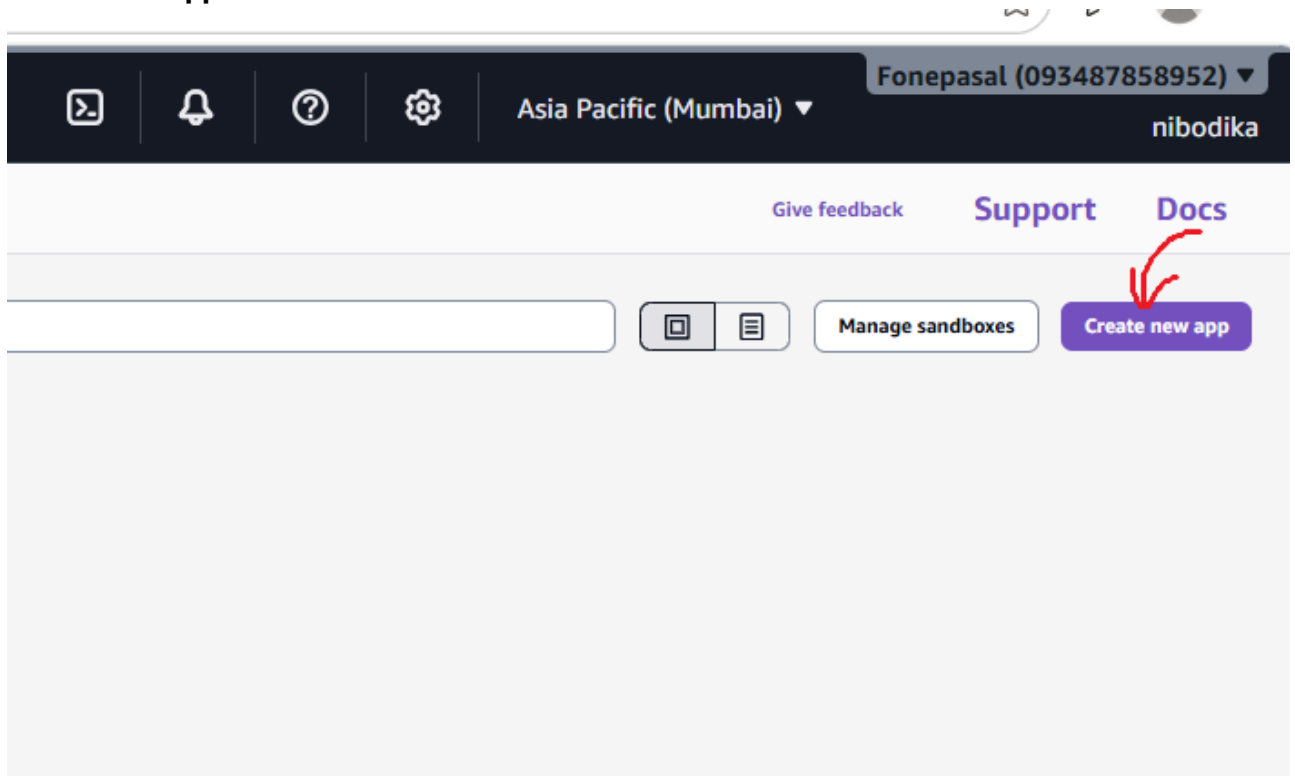
```
git push origin main
```

Step 4: Deploy Using AWS Amplify

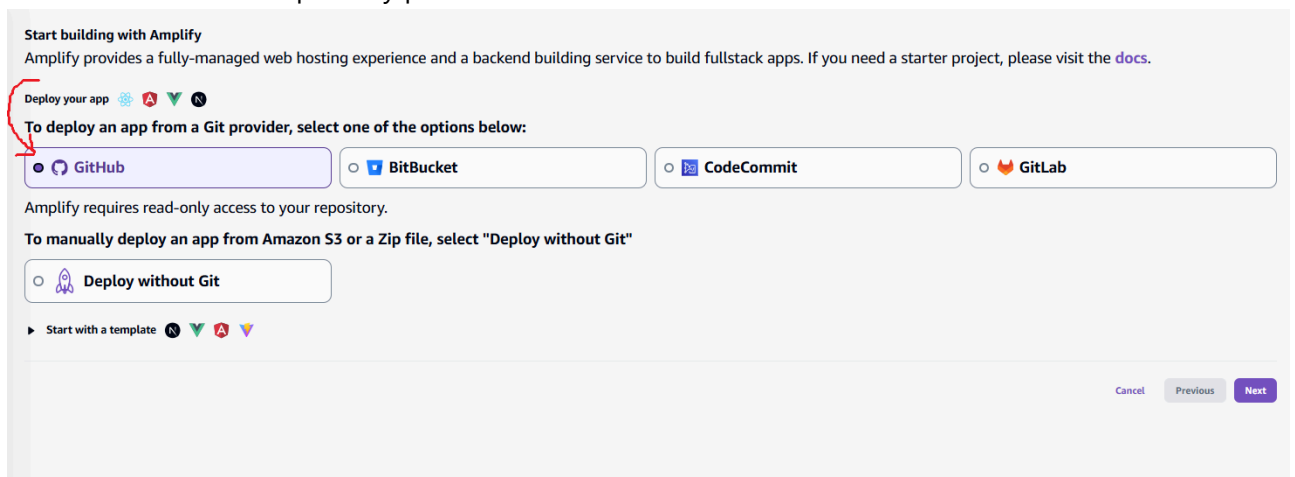
1. Open AWS Console.
2. Navigate to **AWS Amplify**.



3. Click **Create App**.

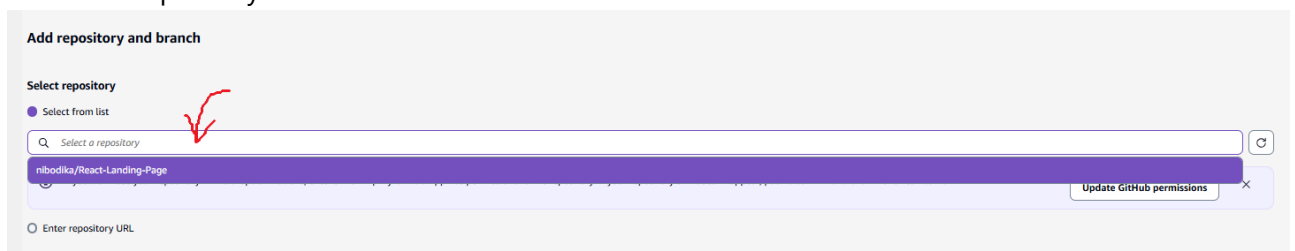


4. Select **GitHub** as the repository provider.



5. Authorize GitHub.

6. Select the repository.



7. Select the branch.

Add repository and branch

Select repository

☒ Select from list

If you don't see your repository in the dropdown above, ensure the Amplify GitHub App has permissions to the repository. If your repository still doesn't appear, push a commit and click the refresh button. [Update GitHub permissions](#)

☐ Enter repository URL

Select branch

main

☐ My app is a monorepo

[Cancel](#) [Previous](#) [Next](#)

8. Click **Next**.

Add repository and branch

Select repository

☒ Select from list

If you don't see your repository in the dropdown above, ensure the Amplify GitHub App has permissions to the repository. If your repository still doesn't appear, push a commit and click the refresh button. [Update GitHub permissions](#)

☐ Enter repository URL

Select branch

main

☐ My app is a monorepo

[Cancel](#) [Previous](#) [Next](#)

9. Review the build settings.

[All apps](#) > **Create new app** [Give feedback](#) [Support](#) [Docs](#)

☒ Choose source code provider

☒ Add repository and branch

☒ **App settings**

☐ Review

App settings

App name

Build settings

Your build settings have been detected automatically, please verify your "Frontend build command" and "Build output directory".

Auto-detected frameworks

Frontend build command Build output directory

[Edit YML file](#)

☐ Password protect my site

[Advanced settings](#) Build instance type, Build image, environment variables, cookies in cache key, live package updates

[Cancel](#) [Previous](#) [Next](#)

Review

Repository details

Repository service: github
Branch: main
Repository: nibodika/React-Landing-Page
Monorepo app root

App settings

App name: React-Landing-Page
Framework: None
Frontend build command: npm run build

Advanced settings

Build instance type: Standard
Build image: Using default image
Keep cookies in cache key: Not enabled
Live package updates
Environment variables: None
Server-Side Rendering (SSR) deployment: Disabled

10. Click **Save and Deploy**.

Cancel **Previous** **Save and deploy**

Amplify automatically:

- Clones the repository
- Installs dependencies
- Builds the application
- Deploys the production build

Amplify Build Process

Amplify executes the following commands automatically.

Install dependencies:

```
npm ci
```

Build application:

```
npm run build
```

Publish:

```
dist/
```

Local Verification

Install dependencies.

```
npm install
```

Run development server.

```
npm run dev
```

Build production version.

```
npm run build
```

Preview production build.

```
npm run preview
```

Deployment Error Encountered

Amplify build failed with the following error:

```
Could not resolve "../styles/navbar.css"  
from  
src/components/Navbar.jsx
```

Root Cause

AWS Amplify runs on **Linux**, which uses a **case-sensitive file system**.

Windows is **case-insensitive**, so the project may build successfully on a local Windows machine even when the filename capitalization does not match the import statement.

Example:

Import statement:

```
import "../styles/navbar.css";
```

Actual filename:

```
Navbar.css
```

This mismatch causes the deployment to fail on Linux.

Solution

Ensure that the filename and the import statement match exactly.

Correct Example:

Filename

```
navbar.css
```

Import

```
import "../styles/navbar.css";
```

or

Filename

```
Navbar.css
```

Import

```
import "../styles/Navbar.css";
```

The capitalization must be identical.

Verify CSS Files

List all files inside the styles directory.

Windows:

```
dir src\styles
```

Linux/macOS:

```
ls src/styles
```

Verify that every CSS import matches the actual filename.

Git Case Rename (If Required)

Git may not detect capitalization-only changes on Windows.

Rename using Git.

```
git mv src/styles/Navbar.css src/styles/temp.css
```

```
git mv src/styles/temp.css src/styles/navbar.css
```

Commit the changes.

```
git add .
```

```
git commit -m "Fix filename casing"
```

Push to GitHub.

```
git push
```

Redeploy the Amplify application.

Common Amplify Build Errors

Module Not Found

```
Could not resolve file
```

Cause:

- Incorrect import path
 - Missing file
 - Incorrect filename capitalization
-

npm Install Failed

```
npm ci failed
```

Possible Causes:

- Missing package-lock.json
 - Invalid package.json
 - Dependency conflict
-

Build Failed

```
vite build failed
```

Possible Causes:

- Syntax error
- Missing assets
- Missing CSS or JavaScript files
- Invalid import statements

Environment Variable Error

```
Failed to set up process.env.secrets
```

Possible Causes:

- Missing Amplify environment variables
- Missing SSM Parameter Store configuration

Best Practices

- Keep filenames lowercase to avoid case-sensitivity issues.
- Verify import paths before deployment.
- Test the production build locally using `npm run build`.
- Commit all project files before deploying.
- Use Git to rename files when changing capitalization.
- Review Amplify build logs after each deployment.

Useful Commands

Install dependencies

```
npm install
```

Run development server

```
npm run dev
```

Build production

```
npm run build
```

Preview production build

```
npm run preview
```

Git status

```
git status
```

Add files

```
git add .
```

Commit

```
git commit -m "Commit message"
```

Push changes

```
git push
```

Conclusion

The React Landing Page was successfully built locally using Vite. During deployment to AWS Amplify, the build failed because of a filename case mismatch (`navbar.css` vs `Navbar.css`). Since AWS Amplify uses a Linux environment, filenames are case-sensitive. Correcting the filename or import statement to match exactly, committing the changes, and redeploying resolves the issue.